

## Homework 2

Name: YOUR NAME; Collaborator: YOUR COLLABORATOR

**Important:** For this homework and all subsequent homeworks, whenever you are asked to analyze a particular algorithm or to design an algorithm, your solution must include a **clear English description** of your thought process. Remember, your audience is a human, not a computer, and we want to understand the process you used to arrive at the solution. This way, we can provide appropriate feedback.

1. You are a backend engineer designing a security feature for a popular web browser. The browser needs to warn users if they attempt to visit a known "malicious" website (e.g., phishing or malware). The database of malicious URLs is massive (containing 10 million entries) and resides on a remote server. To improve speed and privacy, you want to cache a compact version of this database locally on the user's laptop. You decide to use a Bloom Filter for this local cache.
  - (a) In the context of blocking malicious websites, a *false positive* occurs when the user attempts to visit a website which the Bloom filter reports as being "malicious," even though it is not in the set of malicious websites. On the other hand, a *false negative* occurs when the user attempts to visit a website which the Bloom filter reports as being "safe" (not in the set of malicious websites), even though it actually is.
    - Which type of error will *never* occur and which type may occur in a Bloom filter?
    - Which one is merely annoying to the user, and which one a potential security risk?
  - (b) Calculate the required bit-array size  $m$  (in MB), as well as the number of hash functions  $k$ , when the number of items  $n = 10,000,000$  and the target error  $\epsilon = 0.01$  (i.e., %1).
  - (c) Give a comparison for how much smaller the Bloom filter is with respect to the a naive storage of the malicious set of URLs. You may assume that each URL requires 50 bytes to store naively.
  - (d) Six months later, a previously malicious website is cleaned up and is no longer dangerous. It needs to be removed from the blacklist. Can you remove this URL from your standard Bloom Filter? Explain why or why not, referring to how the Bloom filter is constructed.
2. You are a data engineer at a digital advertising firm, and have just run a massive ad campaign across thousands of different websites and mobile apps. For billing purposes, your client doesn't care how many times the ad was shown overall; they only want to pay for the number of *distinct people* who saw the ad.

Due to strict privacy laws (like GDPR or CCPA), the system could not store raw user ids or IP addresses in the database, so the software engineers implemented a the following architecture: ad impressions fly in and the user id is immediately hashed with  $k$  hash functions, inserted into a Bloom Filter of size  $m$ , and immediately discarded. The software engineers designed the Bloom filter so as to support a set of size at most  $n$ , but the true number of distinct

impressions may have been smaller! At the end of the campaign, you are left with a single artifact: the final Bloom Filter bit-array (a long string of 0s and 1s). The client asks: "how many unique people saw the ad?" You know:

- The Bloom filter  $\text{DS} = \{0, 1\}^m$  which we assume is at least 10, and
- $k \in \mathbb{N}$ , the number of hash functions used.

Let  $n_0 \in \mathbb{N}$  be the true (and unknown) number of unique impressions. Your goal is to reverse-engineer the analysis of the Bloom filter in order to find an estimate of  $n_0$ .

- Let  $\mathbf{Z}$  be the random variable given by  $\sum_{j=1}^m \mathbf{1}\{\text{DS}[j] = 0\}$ . Compute the expectation  $\mu = \mathbf{E}[\mathbf{Z}]$  as a function of the true (yet unknown) number of distinct impressions  $n_0$ . Here, we are expecting an expression for  $\mu$  which depends on  $m, k$  and  $n_0$ .

Notice that, while you do not know  $n_0$ , you can compute  $\mathbf{Z}$  from the Bloom filter!

- Use part (a) to "reverse-engineer" an expression, depending on  $\mathbf{Z}, m$  and  $k$  (all quantities that you know!) which gives a potential estimate of  $n_0$  as a function of  $\mathbf{Z}, m$  and  $k$ . The expression should be such that, if  $\mathbf{Z}$  is equal to the expectation, we would exactly recover  $n_0$ . (Here, you may want to recall the definition and rules of working with  $\ln(\cdot)$ ).
  - Use Markov's inequality to conclude that  $\mathbf{Z} \leq 10\mu$  with probability at least 9/10, and as a result, conclude that the estimate you obtain is at least  $n_0 - 3m/k$  with probability 9/10.
3. You are an engineer at a social media platform similar to Twitter (X). The platform processes 100 million tweets per hour. You have been tasked with building a real-time service that tracks the usage frequency of hashtags to identify trending topics. The stream is continuous and the number of possible hashtags enormous, so you cannot store every single hashtag and its count in any standard method. Instead, you choose to use a Count-Min sketch to estimate the frequency of hashtags.
- Give an upper bound on the number of possible hashtags, which we will call  $n$ . Assume each hashtag starts with # and is followed by at most 50 characters of the standard English 26-letter alphabet. Hence, we can conceptually imagine a vector  $x \in \mathbb{R}^n$  where  $x_i$  contains the number of posts with hashtag  $i \in [n]$ . Give a rough estimate on the number of "yottabytes" (highest used unit of storage) that it would take to explicitly store a count for each hashtag, assuming that we devote 8 bytes for each hashtag.
  - The Count-Min sketch will provide an approximate count, which means that for any particular hashtag  $i \in [n]$ , it will output a number  $\hat{x}_i \in \mathbb{R}$ . Decide which of the following is always true, and which may or may not be true:
    - $\hat{x}_i \leq x_i$  for all  $i \in [n]$ .
    - $\hat{x}_i \geq x_i$  for all  $i \in [n]$ .
 Give a brief justification for each.
  - For an accuracy parameter of  $\epsilon = 0.01$ , use Markov's inequality to determine the value of  $m$ , such that a single-hash Count-Min sketch of size  $m$  can estimate, for any fixed hashtag  $i \in [n]$ , the value of  $x_i$  up to an additive error of  $\epsilon\|x\|_1$  with probability at least 0.9.

- (d) Using the median trick, specify the number of times  $\ell$  to repeat the single-hash Count-Min sketch such that for all  $i \in [n]$ ,  $x_i$  may be estimated up to additive error  $\epsilon\|x\|_1$  with probability 0.9.
- (e) Based on your answers to (c) and (d), give an estimate for the total memory in megabytes required, assuming we devote 8 bytes to store numbers.

Your platform scales up, and you now have 3 separate servers ingesting tweets to handle the load. Each server maintains its own local Count-Min sketch which uses the *same* hash functions into  $m$  positions with  $\ell$  repetitions.

- (f) How can you combine these three local sketches into a single global sketch that represents the total counts across all servers?
- (g) Give a brief justification for why it is important that all three servers use the same hash functions.